

深圳大学实验报告

课程名称 机器学习

项目名称 实验六：深度神经网络

学 院 计算机与软件学院

专 业 软件工程（腾班）

指导教师 赖志辉

报 告 人 洪子敬 学号 2022155033

实验时间 2024 年 5 月 20 日至 2024 年 6 月 9 日

实验报告提交时间 2024 年 5 月 29 日

教务处制

一、实验目的与要求

1. 全方面比较 CNN 与全连接网络的异同。
2. 推导神经网络前向后向传播算法的优化迭代公式。
3. 熟练掌握一种深度神经网络的算法与应用，并给出在人脸识别、身份证识别、通用手写体识别等方面的 2 个以上应用实验案例与效果；介绍 transformer 并与它们（1 中的方法）比较识别性能。
4. 参考前人论文，设计一个全新的深度神经网络，使其能在至少一个应用场景能 PK 过经典的 CNN 与全连接网络或现有论文中的算法；简要内容写在实验报告中，详细的可写成一篇论文提交到“论文提交处”。（本点为可选项）

二、实验内容与方法

1. 简要介绍 CNN 与全连接神经网络的原理与两者的异同；
2. 推导全连接神经网络的前向传播和后向传播的优化迭代过程；
3. 用 matlab 实现全连接神经网络并在人脸识别和通用手写字识别上两个案例上测试其效果，计算其精度大小；
4. 用 python 实现 CNN 的经典架构 AlexNet，并在 Fashion_MNIST 数据集上进行训练；
5. 简单介绍 transformer 框架，并比较其与 CNN 的区别；

三、实验步骤与过程

1. 比较卷积神经网络（CNN）与全连接神经网络（FCNN）的异同

卷积神经网络（Convolutional Neural Network, CNN）和全连接神经网络（Fully Connected Neural Network, FCNN）成为图像处理领域的两种重要神经网络模型。

卷积神经网络是从全连接神经网络发展而来的，是一种特殊的深度学习模型，主要用于处理图像数据。卷积神经网络的基本结构包括卷积层、激活函数和池化层。卷积层负责在输入图像上进行局部像素的卷积运算，从而提取出图像的特征；激活函数则用于增加网络的非线性特性；池化层则对特征图进行降维，减少计算量和过拟合的风险。卷积神经网络在图像分类、目标检测、人脸识别等许多任务中都取得了优异的表现。

全连接神经网络是一种传统类型的神经网络，各层之间的节点全部连接。全连接层是全连接神经网络的核心部分，负责从前一层中接收输入并输出到下一层。非线性激活函数在全连接层之间穿插，使得网络具有非线性表达能力。全连接神经网络在处理图像数据时也具有一定的效果，但在对图像特征的提取和表达能力上，相比卷积神经网络略显不足。

CNN 与 FCNN 的相同点：

（1）**结构相似：**卷积神经网络也是通过一层一层的节点组织起来的。和全连接神经网络一样，卷积神经网络中的每一个节点就是一个神经元。在全连接神经网络中，每相邻两层之间的节点都有边相连，于是会将每一层的全连接层中的节点组织成一行，这样方便显示连接结构。而对于卷积神经网络，相邻两层之间只有部分节点相连，为了展示每一层神经元的维度，一般会将每一层卷积层的节点组织成一个三维矩阵。

（2）**训练过程：**卷积神经网络的输入输出以及训练的流程和全连接神经网络也基本一致，以图像分类为例，卷积神经网络的输入层就是图像的原始图像，而输出层中的每一个节点代表了不同类别的可信度。这和全连接神经网络的输入输出是一致的。类似的，全连接神经网络

网络的损失函数以及参数的优化过程也都适用于卷积神经网络。因此，全连接神经网络和卷积神经网络的唯一区别就是神经网络相邻两层的连接方式。

CNN 和 FCNN 的不同点：

- (1) 定义不同：卷积神经网络通过卷积运算提取图像特征，强调局部像素之间的关联性；而全连接神经网络则强调神经元之间的连接，每个神经元与前一层的所有神经元都相连。
- (2) 关键部分不同：卷积神经网络的关键部分包括卷积层、激活函数和池化层，这些部分有助于提取图像特征和处理大规模数据；全连接神经网络的关键部分包括全连接层和非线性激活函数，这些部分使得网络具有非线性特征表达能力
- (3) 应用不同：卷积神经网络在图像处理任务中表现出色，如图像分类、目标检测等，因其能够有效地提取图像特征；而全连接神经网络在处理图像数据时效果也不错，但在特征提取和表达方面相比卷积神经网络略逊一筹。此外，全连接神经网络在文本处理、语音识别等非图像领域也有广泛的应用。

总的来说，卷积神经网络和全连接神经网络在图像处理领域都有一定的应用价值，但两者在结构和应用上存在显著差异。卷积神经网络通过局部像素的卷积运算有效提取图像特征，适用于处理图像数据；而全连接神经网络则强调神经元之间的连接，具有非线性特征表达能力，适用于处理大规模数据。在具体应用中，应根据任务需求和数据特点选择合适的神经网络模型。

2. 全连接神经网络的前向传播和后向传播优化迭代推导

前向传播：（以单层神经网络/感知机为例）

设两个输入神经元与输出层的连接权重分别为 w_1 和 w_2 ，输出层的阈值为 θ ，激活函数设置为 $f(x)$ ，训练数据的原始输出为 y ，那么此例子的前向传播的流程如下：

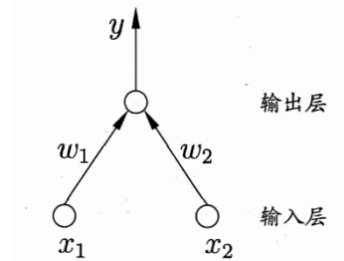
(1) 计算输出层的输入： $input = w_1x_1 + w_2x_2 - \theta$;

(2) 通过激活函数计算输出： $\tilde{y} = f(input)$;

(3) 通过输出结果产生的误差调整权重参数：

$w_i = w_i + \Delta w_i = w_i + \eta(y - \tilde{y})x_i$ ，其中 $\eta \in (0,1)$ ，称为学习率

(4) 直到 w 不再更新，即可停止训练；



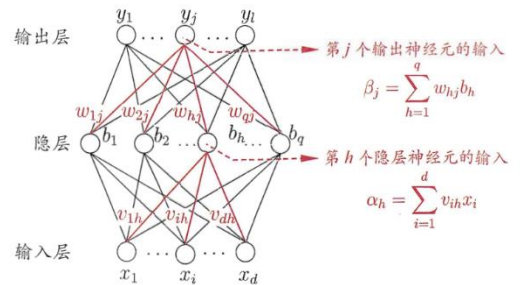
后向传播：（以三层神经网络为例）

设给定数据集 $D = \{(x_1, y_1), (x_2, y_2), \dots, (x_m, y_m)\}$ ，其中 $x_i \in R^d, y_i \in R^l$ ，即输入 d 个属性，输出 l 维向量（一般是不同指标的概率）；同时设输入层第 i 个神经元和隐层第 h 个神经元之间的权重为 v_{ih} ，隐层第 h 个神经元与输出层第 j 个神经元之间的权重为 w_{hj} ，激活函数均采用 $f(x)$ ；则隐层的第 h 个神经元的输入我们记为 $\alpha_h = \sum_{i=1}^d v_{ih}x_i$ ，阈值设为 γ_h ，则其输出为 $b_h = f(\alpha_h - \gamma_h)$ ；

同理，输出层的第 j 个神经元的输入我们记为 $\beta_j = \sum_{h=1}^q w_{hj}b_h$ ，阈值设为 θ_j ，则其输出为 $\tilde{y}_j = f(\beta_j - \theta_j)$ ；

更新过程：（链式法则、梯度下降策略）

我们知道后向传播是先前向传输后计算误差再反向计算梯度，进而更新权重的过程，因此首先我们应该明确此网络在样本 (x_k, y_k) 上的均方误差为：



$$E_k = \frac{1}{2} \sum_{j=1}^l (y_j^k - \hat{y}_j^k)^2$$

对于输出层和隐层之间的权重更新，我们以 w_{hj} 为例进行更新，过程如下：

给定学习率 η ，以目标的负梯度方向对参数进行调整有：

$$\Delta w_{hj} = -\eta \frac{\partial E_k}{\partial w_{hj}}$$

由于 w_{hj} 是先影响到输出层第 j 个神经元的输入 β_j ，再影响到输出层的输出 $\hat{y}_j^k$ ，最后才影响到 E_k ，所以我们使用链式法则则有：

$$\frac{\partial E_k}{\partial w_{hj}} = \frac{\partial E_k}{\partial \hat{y}_j^k} * \frac{\partial \hat{y}_j^k}{\partial \beta_j} * \frac{\partial \beta_j}{\partial w_{hj}}$$

根据我们之前的定义我们可以知道 $\frac{\partial \beta_j}{\partial w_{hj}} = b_h$ ；

为了方便运算我们采用 Sigmoid 函数 ($f(x) = \frac{1}{1+e^{-x}}$) 作为此神经网络的激活函数，则 $f'(x) = f(x)(1-f(x))$ ，这也是 Sigmoid 函数的一个很好的性质，便于我们进行求导；所以有：

$$\frac{\partial E_k}{\partial \hat{y}_j^k} * \frac{\partial \hat{y}_j^k}{\partial \beta_j} = -(y_j^k - \hat{y}_j^k) f'(\beta_j - \theta_j) = -\hat{y}_j^k (1 - \hat{y}_j^k) (y_j^k - \hat{y}_j^k)$$

为了方便记忆，我们记上式为 g_j ，则 $\Delta w_{hj} = \eta g_j b_h$ ；

同理我们对此情况下的阈值 θ_j 来更新，过程如下：

同上在，给定相同学习率 η 下，以目标的负梯度方向对参数进行调整有：

$$\Delta \theta_j = -\eta \frac{\partial E_k}{\partial \theta_j}$$

由于 θ_j 是先影响到输出层第 j 个神经元的输出 $\hat{y}_j^k$ ，进而才影响到 E_k ，所以同理我们使用链式法则则有：

$$\Delta \theta_j = -\eta \frac{\partial E_k}{\partial \theta_j} = -\eta \frac{\partial E_k}{\partial \hat{y}_j^k} * \frac{\partial \hat{y}_j^k}{\partial \theta_j}$$

根据之前的计算我们不难得出： $\Delta \theta_j = -\eta g_j$

类似我们可以计算得到： $\Delta v_{ih} = \eta e_h x_i$ ， $\Delta \gamma_h = -\eta e_h$

值得注意的是，学习率 η 过大容易震荡，太小容易收敛速度过慢，有时为了精细调节，两处输出的 η 使用可能是不一样的值。

在了解前面字母含义的基础上，我们可以给出后向传播的算法流程：

输入：训练集 $D = \{(\mathbf{x}_k, \mathbf{y}_k)\}_{k=1}^m$ ；
学习率 η 。

过程：

- 1: 在(0,1)范围内随机初始化网络中所有连接权和阈值
- 2: **repeat**
- 3: **for all** $(\mathbf{x}_k, \mathbf{y}_k) \in D$ **do**
- 4: 根据当前参数和式(5.3) 计算当前样本的输出 $\hat{\mathbf{y}}_k$;
- 5: 根据式(5.10) 计算输出层神经元的梯度项 g_j ;
- 6: 根据式(5.15) 计算隐层神经元的梯度项 e_h ;
- 7: 根据式(5.11)-(5.14) 更新连接权 w_{hj} , v_{ih} 与阈值 θ_j , γ_h
- 8: **end for**
- 9: **until** 达到停止条件

输出：连接权与阈值确定的多层前馈神经网络

（此处是标准的 BP 算法，只针对单个训练数据进行的更新，此算法虽然容易更快获得局部最优解，但往往需要更多次数的迭代；因此实际当训练集较大时，我们往往采用累积 BP 算法针对累计误差进行最小化）

3. 全连接神经网络 FCNN 的简单编码实现（matlab 实现三层 FCNN）

（1）实现思想：按照全连接神经网络的思想，先对网络的参数进行随机初始化，我们根据输入设置输入和输出神经元数量，并设置隐层数量为 25；随后根据我们之前介绍的网络传播过程设计对应的损失函数，并用划分好的训练集对进行训练得到最终更新后的参数；利用训练好的神经网络模型来预测测试集的结果，计算预测的准确率；

（2）代码实现

下面为核心代码的基本实现：（具体函数请见提交的代码）

```
% 参数初始化
initial_param1=randomInitialWeights(input_layer_size,hidden_layer_size);
initial_param2=randomInitialWeights(hidden_layer_size,num_classified);
% 全部按列排列成列向量
initial_nn_params=[initial_param1(:);initial_param2(:)];
% 训练
options=optimset('MaxIter',350);
lambda=1;
costFunction=@(p)nnCostFunction(p,...
    input_layer_size,...
    hidden_layer_size,...
    num_classified,...
    X_train,Y_train,lambda);
[nn_params,cost]=fmincg(costFunction,initial_nn_params,options);
% 分离权重矩阵
Weight1=reshape(nn_params(1:hidden_layer_size*(input_layer_size+1)),...
    hidden_layer_size,(input_layer_size+1));
Weight2=reshape(nn_params(hidden_layer_size*(input_layer_size+1)+1:end),...
    num_classified,(hidden_layer_size+1));
% 预测
pred=predict(Weight1,Weight2,X_test);
fprintf("Training Set Accuracy:%f\n",mean(double(pred==Y_test))*100);
```

洪子敬 腾班
2022155033

（3）在通用手写字体识别数据集上的测试

我从网上搜寻并下载matlab版本的通用手写字体识别数据集（handwriting.mat），如右图所示：（部分展示）
将其导入后将其按照一定比例划分为训练集和测试集，并进行模型的训练和预测，结果如下：



```
>> main
Training Set Accuracy:93.133333
```

HZJ

（训练集：测试集=7：3）

为进一步测试其性能与划分数数据集比例的关系，我进行了多组实验，实验结果如下所示：

Train与Test的比例	9	4	2.3	1.5	1	0.67
Accuracy/%	92.8	93.5	93.13	92.85	92.24	91.6

由实验结果可知当训练集与测试集的比例为 8：2 的时候实验结果最好，但这也可能也与测试样本数减少有关，所以实际选取比例时我们看到实验结果可以大致判断 6：4 是个较好的划分比例。

此外，先前我们设置的损失函数的最大迭代步数（即最大更新次数）为 350，为进一步探寻

最大迭代步数与实验结果的关系，我们继续进行模型性能与最大迭代步数的关系，实验结果如下所示：（划分比例为 6：4）

MaxIteration	250	350	450	550	650	750	850
Accuracy/%	92.95	92.85	93.4	93.1	93.85	93.35	93.25

由实验结果我们也可以粗略地判断在最大迭代步数 650 时模型可以取得最好的性能。

（4）在人脸数据集上的测试

我们在 AR_Gray_50by40、ORL56_46、Yale_face10080 数据集上进行人脸识别的测试，我们在每个数据集上分别取前 10 类，每类 10 张图片，并按照训练集：测试集=7：3 划分。为了加快模型的训练（因为有些图片的维度比较高，训练时间长），我们采用 PCA 先进行数据的降维，再进行模型的训练，这样可以使得模型更快地捕捉到关键信息完成训练。这里 PCA 是从 20 维开始，以 20 为间隔，叠加到 160。实验结果如下所示：

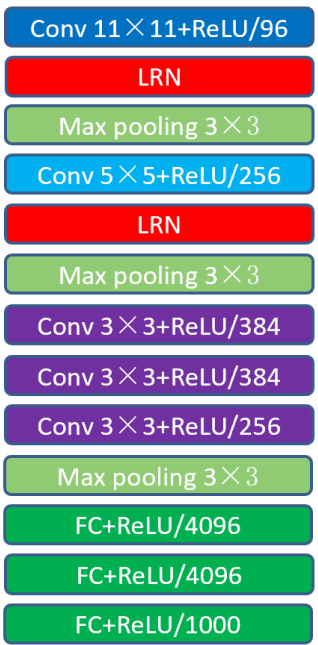
PCA+FCNN在不同维度下对不同人脸数据集上的识别率								
dimension	20	40	60	80	100	120	140	160
AR Gray 50by40	96.67	96.67	96.67	96.67	93.33	96.67	96.67	93.33
ORL56 46	96.67	96.67	96.67	96.67	96.67	1.00	1.00	96.67
Yale face10080	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00

观察实验结果我们可以发现在 AR_Gray_50by40 数据集上最佳识别率是 96.67%，多个维度均可达到；而在 ORL56_46 数据集上，当下降维度是 120 或者 140 的时候，识别率均为最高，达到了 100%；而在 Yale_face10080 数据集上，识别率均可达到 100%，这一方面与数据集有关，另一方面和神经网络的优势有关。总而言之，全连接神经网络在人脸识别上整体的性能还是不错的。

4. 经典深度模型 AlexNet 的介绍和 python 实现

（1）简要介绍：LeNet 是最早的深度卷积神经网络模型，实现也较为简单，AlexNet 在其基础上进行了更宽更深的网络设计，首次在 CNN 中引入了 ReLU、Dropout 和 Local Response Norm (LRN)等技巧。这些技术特点具体介绍如下：（模型架构如右图所示）

- ①使用 ReLU 作为 CNN 的激活函数，并验证其效果在较深的网络超过了 Sigmoid，成功解决了 Sigmoid 在网络较深时的梯度弥散问题，提高了网络的训练速率；
- ②为避免过拟合，训练时使用 Dropout 随机忽略一部分神经元；
- ③使用重叠的最大池化(max pooling)。最大池化可以避免平均池化的模糊化效果，而采用重叠技巧可以提升特征的丰富性；
- ④提出了 LRN 层 (ReLU 后进行归一化处理)，对局部神经元的活动创建竞争机制，使得其中响应比较大的值变得相对更大，并抑制其他反馈较小的神经元，增强了模型的泛化能力；
- ⑤利用 GPU 强大的并行计算能力加速网络训练过程，并采用 GPU 分块训练的方式解决显存对网络规模的限制；
- ⑥采用数据增强技术。利用随机裁剪和翻转镜像操作增加训练数据量，降低过拟合风险。



(2) python 实现：我们使用 pytorch 工具和经典的深度学习 d2l 库对此架构进行实现，具体实现时我们将 dropout 设置为 0.5，并把输出结果设置为 10 类（因为我们用来训练和测试的数据集一共就 10 类），架构核心代码如下所示：

```
import torch
from torch import nn
from d2l import torch as d2l
洪子敬 腾班
2022155033

net = nn.Sequential(
    nn.Conv2d(1, out_channels=96, kernel_size=11, stride=4, padding=1), nn.ReLU(),
    nn.MaxPool2d(kernel_size=3, stride=2),
    nn.Conv2d(96, out_channels=256, kernel_size=5, padding=2), nn.ReLU(),
    nn.MaxPool2d(kernel_size=3, stride=2),
    nn.Conv2d(256, out_channels=384, kernel_size=3, padding=1), nn.ReLU(),
    nn.Conv2d(384, out_channels=384, kernel_size=3, padding=1), nn.ReLU(),
    nn.Conv2d(384, out_channels=256, kernel_size=3, padding=1), nn.ReLU(),
    nn.MaxPool2d(kernel_size=3, stride=2),
    nn.Flatten(),
    nn.Linear(6400, out_features=4096), nn.ReLU(), nn.Dropout(=0.5),
    nn.Linear(4096, out_features=4096), nn.ReLU(), nn.Dropout(=0.5),
    nn.Linear(4096, out_features=10)
)
```

接着我们采用 fashion_mnist 数据集对其进行训练和测试如下所示：

（由于此数据集每张图片大小都是 28*28，而为了与 ImageNet 数据集大小对齐，我们将其拉伸至 224*224，但实际对于其他训练不会这么用，因为实际并没有什么效果）

```
X = torch.randn(1, 1, 224, 224)
洪子敬 腾班
2022155033
for layer in net:
    X = layer(X)
    print(layer.__class__.__name__, 'Output shape:\t', X.shape)

# 使用Fashion-MNIST图像，分辨率为28*28，需增加至284*284
batch_size = 128
train_iter, test_iter = d2l.load_data_fashion_mnist(batch_size, resize=224)

lr, num_epochs = 0.1, 10
d2l.train_ch6(net, train_iter, test_iter, num_epochs, lr, d2l.try_gpu())
```

我们为了方便看到每一步卷积的结果，我们打印出每次的结果图像大小，结果如下所示：

```
C:\Users\28867\conda\envs\PyTorch\python.exe D:\DeepLearning\d2l\AlexNet.py
Conv2d Output shape: torch.Size([1, 96, 54, 54])
ReLU Output shape: torch.Size([1, 96, 54, 54])
MaxPool2d Output shape: torch.Size([1, 96, 26, 26])
Conv2d Output shape: torch.Size([1, 256, 26, 26])
ReLU Output shape: torch.Size([1, 256, 26, 26])
MaxPool2d Output shape: torch.Size([1, 256, 12, 12])
Conv2d Output shape: torch.Size([1, 384, 12, 12])
ReLU Output shape: torch.Size([1, 384, 12, 12])
Conv2d Output shape: torch.Size([1, 384, 12, 12])
ReLU Output shape: torch.Size([1, 384, 12, 12])
Conv2d Output shape: torch.Size([1, 256, 12, 12])
ReLU Output shape: torch.Size([1, 256, 12, 12])
MaxPool2d Output shape: torch.Size([1, 256, 5, 5])
Flatten Output shape: torch.Size([1, 6400])
Linear Output shape: torch.Size([1, 4096])
ReLU Output shape: torch.Size([1, 4096])
Dropout Output shape: torch.Size([1, 4096])
Linear Output shape: torch.Size([1, 4096])
ReLU Output shape: torch.Size([1, 4096])
Dropout Output shape: torch.Size([1, 4096])
Linear Output shape: torch.Size([1, 10])
洪子敬 腾班
2022155033
```

而当我们设置学习率为 0.01 时，其训练和测试结果如下所示：（设备为 cpu，耗时 1.5h）

```
loss 0.328, train acc 0.880, test acc 0.882
68.0 examples/sec on cpu
洪子敬 腾班
2022155033
Process finished with exit code 0
```

当我们设置学习率为 0.1 时，其训练和测试结果如下所示：（设备为 cpu，耗时 2.0h）

```
loss 0.182, train acc 0.932, test acc 0.916
67.6 examples/sec on cpu
洪子敬 腾班
2022155033
Process finished with exit code 0
```

对比两次结果，我们可以发现当学习率设置为 0.1 时效果会更加，学习率也似乎可以继续测试提升，但过拟合风险也会进一步提升；由于设备不支持，本人并未继续往下做实验。

5. Transform 简介

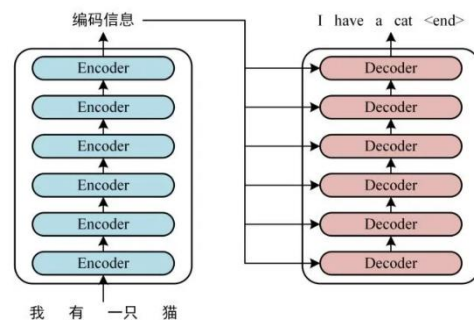
为了方便理解 Transformer 的核心概念，我们以其用于中英文翻译的整体结构：

如右图所示，其主要由 Encoder 和 Decoder 两部分组成，每个部分都包含 6 个 block。其工作流程组要如下：

(1) 获取输入句子的每一个单词表示向量 X , X 由单词的 Embedding (集从原始数据中提取出来的 features) 和单词位置的 Embedding 相加得到；

(2) 将得到的单词表示向量组成矩阵，传入 Encoder 中，经过 6 个 block 后可以得到句子所有单词的编码信息矩阵 ($n \times d$, n 为单词数, d 为向量维度)。每个 Encoder block 输出的矩阵维度与输入完全一致；

(3) 将 Encoder 输出的编码信息矩阵 C 传递到 Decoder 中，Decoder 依次会根据当前翻译过的单词 $1-i$ 来翻译下一个单词 $i+1$ ，此过程需要掩盖 (Mask) $i+1$ 之后的单词；通过这三步，中文基本就可以翻译成英文。



而每个编码器里面均含有两部分：自注意力机制和前馈神经网络，每个解码器里面包括三部分：自注意力机制，编码-解码注意力层以及前馈神经网络。具体可见下图所示：

自注意力机制这里简单地介绍一下，具体计算流程（特别是 K 、 Q 、 V ）不加叙述：

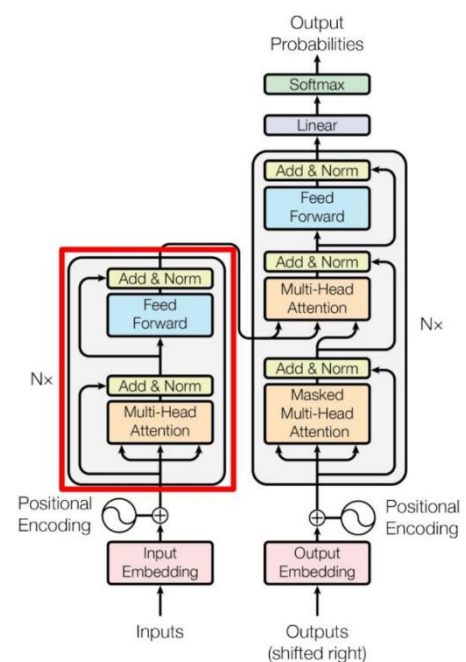
其作用：随着模型处理输入序列的每个单词，自注意力会关注整个输入序列的所有单词，帮助模型对本单词更好地进行编码。在处理过程中，自注意力机制会将对所有相关单词的理解融入到我们正在处理的单词中。

自注意力机制的具体功能：

(1) 序列建模：自注意力可以用于序列数据（例如文本、时间序列、音频等）的建模。它可以捕捉序列中不同位置的依赖关系，从而更好地理解上下文。这对于机器翻译、文本生成、情感分析等任务非常有用；

(2) 并行计算：自注意力可以并行计算，这意味着可以有效地在现代硬件上进行加速。相比于 RNN 和 CNN 等序列模型，它更容易在 GPU 和 TPU 等硬件上进行高效的训练和推理。（因为在自注意力中可以并行的计算得分）；

(3) 长距离依赖捕捉：传统的循环神经网络 (RNN) 在处理长序列时可能面临梯度消失或梯度爆炸的问题。自注意力可以更好地处理长距离依赖关系，因为它不需要按顺序处理输入序列。



在经过多头注意力机制得到矩阵之后，并没有直接传入全连接神经网络，而是经过了一步 Add&Normalize，这一步可以分为 Add 和 Norm 两部分：

Add，就是在原矩阵的基础上加了一个残差块 X ，加入残差块的目的是为了防止在深度神经网络的训练过程中发生退化的问题，退化的意思就是深度神经网络通过增加网络的层数，Loss 逐渐减小，然后趋于稳定达到饱和，然后再继续增加网络层数，Loss 反而增大；

Norm，归一化，目的是为了加快训练速度同时提高训练的稳定性；

最后的输出首先经过一次线性变换（线性变换层是一个简单的全连接神经网络，它可以把解码组件产生的向量投射到一个比它大得多的，被称为对数几率的向量里），然后 Softmax 得到输出的概率分布（softmax 层会把向量变成概率），然后通过词典，输出概率最大的对应的单词作为我们的预测输出。

（具体实现，由于细节较多，这里不多加叙述；但值得承认的是 transformer 架构能很好地进行并行训练，不同于以往的 RNN，且引入了自注意力机制且使用了多层，能够很好的捕捉单词之间的联系，效果很不错）

五、实验结论或体会

实验结论：

本次实验学习了卷积神经网络、全连接神经网络甚至对 Transformer 架构进行了初步的理解，也区分了 CNN 和 FCNN 之间的异同点，还对前向传播和后向传播的原理进行了推导，进一步加深了对神经网络的理解；此外，还使用 matlab 实现了全连接神经网络，对神经网络的优化过程有了进一步的理解；同时我们还是在通用手写字体识别与人脸识别两个案例上测试了该神经网络的性能，试验结果表明其不错的性能；最后我们还实现了 AlexNet 框架，第一次使用深度卷积神经网络模型，成功在 fashion_mnist 数据集上训练和测试了该模型，性能取得不错效果；本次实验圆满结束。

实验体会：

本实验最大的收获就是深入地学习了全连接神经网络、卷积神经网络的原理以及前向传播、后向传播地优化过程，已经对神经网络有了初步的掌握，为之后的学习更加深层的神经网络打下比较好的基础。

此外，本实验采用 matlab 编程，在编写全连接神经网络的代码时，有些部分都是不熟悉的编程吗，都是去网上查找相应的用法才得以解决，最困难的部分还是损失函数部分、优化迭代过程的代码编写，采用一些 matlab 自带的许多功能，在体会到 matlab 便捷的同时也长了见识，学习到许多东西；

另外，本次实验我也第一次调用各种库和函数实现了经典深度卷积网络模型 AlexNet，还对其进行了训练和测试，初步体会到了“炼丹”过程，也取得了较好的效果；可惜个人设备不支持，跑的时间太长了，以后有条件再多试试；

最后实验过程中也因为学业繁忙读的论文不是很多，没有去思考创新的神经网络模型，同时 transformer 架构也是第一次接触到，不是很能理解原理之类的内容，后续也需要自己不断去学习深度学习的内容，争取早日理解。

指导教师批阅意见:

成绩评定:

指导教师签字:

年 月 日

备注:

注：1、报告内的项目或内容设置，可根据实际情况加以调整和补充。